TP1 : Git

Partie 1 : Récapitulatif

1. Créez un dossier «TP1 » et Positionnez-vous sous ce dossier.
2. Initialisez le git sous ce dossier pour qu'il soit suivi par ce dernier.
3. Listez la liste des éléments existants sous ce répertoire pour vérifier l'existence du dossier .git.
4. Créer un fichier file1.txt sous le répertoire « TP1 » et écrivez dedans.
5. Afficher le contenu du fichier créé.
6. Vérifiez l'état du git avec la commande ***« git status »***. On remarque l'existence de 3 zones :
   - **Branch Master (main):** branche principale.
   - **Untracked Area** : contenant tous les fichies non encore suivis par le git.
   - **Staging Area :** espace brouillant où le git suit les modifications des fichiers tracked.
7. Tapez la commande ***« git add file1.txt »*** pour suivre les modifications apportées au niveau de ce fichier par le git. Ainsi, le fichier file1.txt devient tracked. Le fichier file1.txt passe maintenant du ***untracked area*** vers ***staging area***
   Si vous voulez suivre tous les fichiers existants sous ce dossier par le git il suffit de taper la commande ***« git add . »***
8. Faites un commit pour sauvegarder les modifications avec la commande ***« git commit -m "Nom-du-commit " ».***
9. Créez deux fichiers file2.txt et file3.txt, remplissez-les et faites un commit.
10. Listez les commits effectués avec la commande ***« git log ».***
11. Listez les 2 derniers commits ***« git log –nbre ».***
12. Lister les commit effectués alignés selon leurs ids ***« git log –oneline ».***

Remarque : Afin d'avoir une traçabilité du travail, il est nécessaire de configurer les informations de chaque utilisateur.

13. Configurez les informations utilisateur concernant le username et l'email avec les commandes ***« git config –global user.name "Nom-utilisateur"»*** et ***« git config –global user.email "email-utilisateur"»***.
14. Créez un nouveau fichier file4.txt, commitez-le et consulter le log pour vérifier la configuration de l'utilisateur effectuée.

Partie 2 : GitHub

L'objectif de cette partie est de déposer notre travail en remote. Pour ce faire, on va utiliser GitHub qui est un service à base de cloud aidant les développeurs à stocker, suivre et gérer leurs codes. Dans notre cas, il va jouer le rôle d'un repository distant.

1. Créer tout d'abord un compte sur le site GitHub sur ce lien *https://github.com/login*.
2. Créez un repository « First_Project » en le configurant public.
3. A quoi ça sert le répertoire « .gitignore ».

4. Créez un fichier sous le repository distant « FirstFile », remplissez-le et faites un commit sur le repository.
5. Faites un clonage (clone) du projet sur votre machine locale. Allez vers le bouton *Code -> HTTPS -> copier le lien* et lancer sur le terminal la commande ***« git clone url_copié ».***
   ⇨ A travers cet url, on pourra accéder à notre projet distant via le terminal.
6. Vérifier que le projet est bien cloné en local.
7. Positionnez-vous sous le projet cloné.
8. Consulter la liste des fichiers existants sous ce répertoire.
9. Créez un nouveau fichier en local « LocalFile » et remplissez-le.
10. Ajoutez le fichier pour être suivi par le git et faites un commit.
11. Vérifiez que le fichier est tracked et afficher le log.
12. Vérifiez l'existence du fichier sur le repository distant. Le fichier LocalFile existe-il ? Pourquoi ?
13. Afin d'arriver à avoir les modifications apportées au niveau du projet local sur le repository distant, tapez la commande ***« git push ».***
14. Que se passe-il ? Pourquoi ?

L'authentification par username et password n'est plus prise en charge par GitHub depuis 13/09/21. Une méthode plus sécurisée est utilisable est l'authentification par tokens ». La tokenisation consiste à donner des ***privilèges spécifiques*** *à des* ***utilisateurs spécifiques*** *pour une* ***période précise****.*

15. Pour générer un token sur GitHub, allez vers le ***logo user -> settings -> Developper settings***
    -> Personal access tokens -> Generate new token.
    Remplissez le nécessaire : nom du token, les privilèges, les ressources allouées, date d'expiration du token, …
16. Après avoir généré le token, sauvegardez le dans un fichier à part.
17. Retapez la commande ***« git push »*** et faites entrer le username et le token déjà généré.
18. Consultez maintenant que le repository distant est mis à jour. (le fichier local est uploadé)
19. Maintenant, créez un nouveau fichier sur le repository distant. Faites le nécessaire pour avoir les modifications apportées en local, avec la commande ***« git pull »***.
20. Pourquoi il n'a pas demandé une authentification, contrairement à l'action de push.

Une autre méthode d'authentification est la méthode par SSH (Secure Shell Protocol) permettant la connexion à une machine distante via une paire de clé privé / publique. Pour renforcer de plus la sécurité par SSH, on pourra rajouter une passphrase qu'on fournit lors de la connexion.

21. Pour générer une clé SSH, tapez la commande ***« ssh -keygen -t ed25519 ».*** On pourra afficher la clé publique en tapant la commande ***« cat chemin_généré ».***

22. Copiez la clé générée et allez vers ***settings->SSH and GPG keys -> New SSH Key***. Sélectionnez un nom pour votre clé et collez la clé déjà copiée.
23. Vérifiez la connexion SSH. Tapez la commande ***«ssh -T git@github.com »***.
24. Supprimer le projet «First_Project » en tapant la commande ***« rm -rf First_Project »***.
25. Copiez l'url SSH du projet et faites un clonage du projet de nouveau.
26. Vérifiez que le projet est cloné.